

Week 2: Docker 기초와 실행 환경 표준화

Overview

2주차는 Week 1에서 만든 로컬 실행 evidence를 Docker 이미지와 컨테이너로 옮기는 주간이다. 학생은 "내 컴퓨터에서는 실행된다"를 넘어서, 같은 실행 조건을 다른 사람도 재현할 수 있도록 Dockerfile, image, container, port binding, volume, environment variable, Compose로 정리한다.

이번 주의 중심 질문은 다음과 같다.

애플리케이션 실행 조건을 이미지와 컨테이너로 표준화하면 실행, 확인, 중지, 복구, 인수인계가 어떻게 달라지는가?

Docker는 운영 문제를 모두 없애는 도구가 아니다. 실행 환경을 더 명시적으로 포장하고, 실행 조건을 문서와 명령으로 재현 가능하게 만드는 도구다. 따라서 이번 주의 평가는 명령어 암기가 아니라 실행 증거, 장애 기록, README handoff, 비용/보안/정리 기준을 기준으로 한다.

처음이면 여기부터

Week 2는 Docker가 반드시 필요하다. macOS는 Docker Desktop을 기본 경로로 진행하고, Linux는 Docker Desktop 또는 Docker Engine 중 하나를 선택한다. 설치가 끝난 뒤에는 "앱을 열었다"가 아니라 CLI evidence로 확인한다.

- macOS/Linux 설치 절차: [필수 소프트웨어 설치 가이드](#)
- Docker 최소 확인 명령: `docker version`, `docker compose version`, `docker run --rm hello-world`
- Linux에서 권한 문제가 나면 `sudo docker run --rm hello-world` 성공 여부와 Docker post-install 적용 여부를 구분해 기록한다.
- Docker Hub password, access token, MFA code는 README, terminal history, screenshot에 남기지 않는다.

Learning Goals

- Docker가 해결하려는 실행 환경 차이, 배포 재현성, 의존성 충돌 문제를 설명한다.
- image와 container의 차이를 파일 묶음과 실행 중인 process 관점으로 구분한다.
- `docker run`, `docker ps`, `docker logs`, `docker stop`, `docker rm`으로 컨테이너 lifecycle을 확인한다.
- PostgreSQL 16/18 container를 서로 다른 host port로 실행해 version isolation과 port binding을 검증한다.
- PostgreSQL container 안에서 user, database, table, row를 만들고 query로 정상 동작을 확인한다.

- volume 없이 container를 재생성했을 때 데이터가 사라지는 이유를 설명한다.
- named volume을 database container에 연결해 container 교체 후에도 데이터가 유지되는지 확인한다.
- port binding, environment variable, volume, network, registry/image tag를 이용해 컨테이너 실행 조건을 제어한다.
- logs, inspect, exec, stats로 container 상태를 증거 기반으로 확인한다.
- Docker Compose로 유명한 로컬 아키텍처 패턴을 실행하고 검증한다.
- 로그, HTTP status, container status, README 기록으로 장애 원인을 분석한다.

Weekly Keywords

- Docker
- image
- container
- Docker Desktop
- Docker Engine
- registry
- Docker Hub
- Dockerfile
- layer
- tag
- digest
- port binding
- volume
- bind mount
- named volume
- environment variable
- bridge network
- Docker Compose
- service
- compose.yaml
- container logs

Schedule Index

- Day 1: 짧은 Week 1 리뷰, Docker 공식 컨셉, macOS/Linux 설치, nginx 실행 확인, 로컬 PostgreSQL 충돌/삭제, PostgreSQL 16/18 병렬 실행, SQL 기본 조작, 개인 면담
- Day 2: Storage and network - volume persistence, bind mount, custom bridge network, container name DNS, host port publish 차이 실험
- Day 3: Image, Dockerfile, registry - layer/cache/tag/digest, build context, 표준 앱 build/run, Docker Hub와 push gate

- Day 4: Runtime config and observability - env/env-file, secret 비노출, logs/inspect/exec/stats, failure drill, cleanup/security audit
- Day 5: Docker Compose 확정 - 제공 코드와 compose.yaml로 Web+DB, DB UI, cache, reverse proxy, queue+worker 같은 유명 아키텍처를 실행/검증

Week 2 Day 역할 구분

Day	핵심 역할	중복 방지 기준
Day 1	Docker 설치와 기본 실행 가능 여부를 nginx로 확인하고, PostgreSQL을 Docker로 띄우며 port 충돌, version 분리, SQL 기본 조작을 검증	volume persistence는 깊게 다루지 않고, 7~8교시는 면담과 환경 점검으로 사용
Day 2	storage와 network를 깊게 다룬다	volume persistence, bind mount, custom network, container DNS, host publish 차이를 실험으로 분리
Day 3	image와 build/registry를 깊게 다룬다	Dockerfile, build context, layer/cache, tag/digest, official image와 push gate를 한 흐름으로 묶음
Day 4	runtime config와 observability/troubleshooting을 깊게 다룬다	env/secret, logs/inspect/exec/stats, failure drill, cleanup/security audit로 Compose 전 운영 감각 확보
Day 5	Compose 확정: 제공 코드로 유명한 로컬 아키텍처 패턴을 여러 개 실행한다	YAML 설명으로 끝내지 않고 config/up/ps/logs/curl/exec/down으로 실제 동작을 검증

강의 본문에는 분 단위 흐름표를 넣지 않는다. 시간 배분은 강사용 참고로만 사용하고, 학생용 자료는 개념 전개, 실습 명령, expected evidence, 흔한 오해, cleanup 기준을 중심으로 작성한다.

CLI로 실행할 명령은 반드시 fenced code block으로 작성한다. 각 실습은 실행 명령, 검증 명령, 실패 판정, cleanup 명령을 함께 제공하고, 강사는 사전에 macOS 또는 Linux에서 명령을 실행해 실제 동작 여부를 확인한다.

Week 1 To Week 2 Mapping

Week 1 local evidence	Week 2 Docker 표현	운영 판단
app folder	build context, COPY 범위	이미지에 필요한 파일만 포함했는가
start command	CMD, docker run command	컨테이너가 어떤 process로 시작되는가
localhost port	-p host:container port binding	외부 사용자가 어느 port로 접근하는가
data path	bind mount, named volume	컨테이너 삭제 후 데이터가 남아야 하는가
config note	-e, .env, Compose environment	설정을 이미지에 굳히지 않았는가
log evidence	docker logs, docker compose logs	장애 증거를 어디서 확인하는가
README run step	Docker build/run/compose section	다른 사람이 실행 절차를 재현할 수 있는가
RCA note	Docker failure analysis	port conflict, env missing, volume reset을 기록했는가

Required Deliverables

- 표준 실습 애플리케이션용 Dockerfile 또는 Day 3~4 확정 후 대체 build artifact
- compose.yaml 1개
- Docker build/run/compose 실행 명령이 포함된 README.md
- Docker Hub 또는 표준 registry에서 내려받아 실행한 이미지 1개
- PostgreSQL 16/18 port binding 실험 evidence 1개
- PostgreSQL SQL 조작 evidence 1개: user, database, table, insert, query
- storage/network evidence 2개 이상
- image/Dockerfile/registry evidence 2개 이상
- env/logs/inspect/exec failure drill evidence 1개 이상
- Compose architecture 실행 evidence 2개 이상
- port, log, environment variable, volume, network 중 하나 이상을 포함한 장애 분석 기록 1개
- 3주차 학습 전 Docker readiness checklist

Assessments

- [Week 2 객관식 문제 세트](#)

Practice Environment

항목	기준
OS	macOS와 Linux를 기본 경로로 분리. Windows 사용자는 WSL 2/가상화/권한 예외 경로를 별도 기록
Docker	macOS는 Docker Desktop, Linux는 Docker Desktop 또는 Docker Engine 중 하나를 공식 문서 기준으로 설치/확인
CLI	<code>docker version</code> , <code>docker run hello-world</code> , <code>docker ps</code> 실행 가능
PostgreSQL client	host <code>psql</code> 이 있으면 host port 접속 확인, 없으면 <code>docker exec ... psql</code> 로 version query
Repository	Dockerfile, compose, README, RCA note를 저장할 GitHub repository
비용	Week 2는 cloud resource를 만들지 않는다. 로컬 Docker와 Docker Hub pull/push 중심으로 진행한다.

Official References

Topic	Reference	확인할 키워드
Docker overview	https://docs.docker.com/get-started/docker-overview/	client-server architecture, daemon, registry, images, containers
Docker Desktop	https://docs.docker.com/desktop/	Desktop, GUI, build/share/run
Mac install	https://docs.docker.com/desktop/setup/install/mac-install/	Apple silicon, Intel, system requirements
Linux Desktop install	https://docs.docker.com/desktop/setup/install/linux/	Linux VM, Docker context, host Engine 차이
Ubuntu Engine install	https://docs.docker.com/engine/install/ubuntu/	Docker Engine, apt repository, daemon
Windows install	https://docs.docker.com/desktop/setup/install/windows-install/	Windows 사용자의 WSL 2, system requirements, permissions
Container concept	https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/	isolated process, VM comparison, port exposure
Image concept	https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/	immutable, layers, package
PostgreSQL official image	https://github.com/docker-library/docs/blob/master/postgres/README.md	POSTGRES_PASSWORD, PGDATA, tags
Dockerfile	https://docs.docker.com/guides/docker-concepts/building-images/writing-a-dockerfile/	FROM, WORKDIR, COPY, RUN, CMD
Docker Compose	https://docs.docker.com/compose/	multi-container, services, networks, volumes
Compose services	https://docs.docker.com/reference/compose-file/services/	ports, environment, volumes, networks
Compose networking	https://docs.docker.com/compose/how-tos/networking/	service name, DNS, network

Cost And Security Notes

- Week 2 실습은 로컬 Docker 중심이므로 AWS, Kubernetes cluster, 유료 cloud database를 만들지 않는다.
- Docker Desktop 라이선스 조건은 조직 규모와 사용 목적에 따라 달라질 수 있으므로 공식 문서와 교육 운영 기준을 확인한다.

- Docker Hub password, access token, MFA code는 README, screenshot, terminal history에 남기지 않는다.
- DB password는 실습용이어도 공개 repository에 그대로 올리지 않는다. 공개해야 할 것은 값이 아니라 "어떤 환경변수 이름이 필요한가"와 "어디에 기록하지 않는가"다.
- 실습 후 불필요한 container, image, volume을 정리해 host disk 사용량을 관리한다.

Weekly Checklist

- [] docker version 결과를 기록했다.
- [] docker run hello-world 성공 또는 실패 증상을 기록했다.
- [] Week 1 로컬 PostgreSQL을 삭제/중지/보류 중 하나로 처리하고 이유를 기록했다.
- [] postgres:16과 postgres:18을 서로 다른 host port로 실행하거나 실패 증상을 기록했다.
- [] 같은 host port 충돌을 재현하고 port binding 관점으로 설명했다.
- [] PostgreSQL container에서 user/database/table/row를 만들고 query 결과를 기록했다.
- [] volume 없이 container를 재생성하면 데이터가 사라지는 것을 확인했다.
- [] named volume을 연결하면 container 교체 후에도 데이터가 유지되는 것을 확인했다.
- [] custom network에서 container name으로 통신했다.
- [] Dockerfile로 표준 앱 image를 build/run하고 HTTP 확인했다.
- [] official image tag/digest 또는 docker image inspect 결과를 기록했다.
- [] env/config 주입, logs/inspect/exec, secret 비노출 기준을 기록했다.
- [] compose.yaml로 최소 2개 아키텍처를 실행하고 검증했다.
- [] logs/status/HTTP response 중 하나 이상으로 정상 상태를 확인했다.
- [] port conflict, env missing, volume issue 중 하나를 RCA 형식으로 기록했다.

Glossary

2주차 용어는 [glossary.md](#)를 기준으로 정리한다. 용어 정의는 명령어 암기보다 "어떤 상태를 확인하거나 어떤 운영 위험을 줄이는가"에 연결해 읽는다.

Next Week Connection

3주차의 MSA는 여러 서비스가 API와 network로 연결되는 구조를 다룬다. Week 2에서 Compose로 웹 앱과 DB를 함께 실행해 본 경험은 서비스 분리, service name, network boundary, configuration handoff를 이해하는 선행 경험이 된다.